

Databases & MongoDB Lecture Notes

1. What is a Database and Why Do We Need It?

Meaning: A database is an organized collection of structured information, or data, typically stored electronically in a computer system.

Why we need it: If we save data in simple variables or arrays in our code, that data disappears the moment the server shuts down or restarts. Databases allow us to store data permanently (persistence), search through it quickly, and handle thousands of users simultaneously without crashing.

Types of Databases

Databases are generally split into two main categories: SQL (Relational) and NoSQL (Non-Relational).

1. SQL (Relational Databases):

- How they work: Data is stored in strict, predefined Tables with Rows and Columns (like an Excel spreadsheet).
- Examples: MySQL, PostgreSQL, Oracle.
- Best used for: Complex, highly structured data where relationships between tables are strict (e.g., banking systems).

2. NoSQL (Non-Relational Databases):

- How they work: Data is stored in flexible, dynamic formats, often resembling JSON documents.
- Examples: MongoDB, Redis, Cassandra.
- Best used for: Rapid development, scaling large amounts of unstructured data, and real-time web apps.

2. ACID vs. BASE Properties

ACID (Relational / SQL):

- Atomicity: "All or nothing." If one part fails, the whole transaction fails.
- Consistency: Data must always follow the defined rules/schemas.
- Isolation: Multiple concurrent transactions don't interfere with each other.

- Durability: Once saved, it's permanent.

BASE (NoSQL / MongoDB):

- Basically Available: Guarantees availability over absolute consistency.
- Soft state: The state of the system may change over time.
- Eventual consistency: Given enough time, all copies of the data will become consistent.

3. Introduction to MongoDB

Meaning: MongoDB is the most popular NoSQL database. Instead of tables and rows, it stores data in flexible, JSON-like documents.

Why MongoDB?

1. Flexibility: You don't have to define a strict schema before inserting data.
2. JavaScript Native: It uses JSON-style syntax, meaning JavaScript developers can use it immediately without learning SQL.
3. Scalability: Built to scale horizontally easily.

SQL vs MongoDB Terminology

- SQL Database === MongoDB Database
- SQL Table === MongoDB Collection
- SQL Row === MongoDB Document
- SQL Column === MongoDB Field

4. The MongoDB Structure

Hierarchy: Database -> Collection -> Document -> Field

Code:

```
// Example of a single Student Document
{
  "_id": ObjectId("60c72b2f9b1d8b3a4c123456"),
  "name": "Alice",
  "age": 22,
  "courses": ["Math", "Computer Science"], // Arrays
  "address": {                               // Nested
    "city": "New York",
    "zip": "10001"
  }
}
```

```
}  
}
```

5. BSON and ObjectIds

BSON (Binary JSON): MongoDB converts JSON into BSON before saving. It is faster for the computer to parse and adds support for extra data types like Dates and ObjectIds.

ObjectId (`_id`): A 12-byte unique alphanumeric string automatically generated by MongoDB if you don't provide an ID. It acts as the Primary Key.

6. MongoDB Installation Guide

1. MongoDB Community Edition (The Engine): Download from the MongoDB website. Install as a background service.

2. MongoDB Compass (The GUI): A visual tool to look inside your database without the command line. Connect using the default URI: ``mongodb://localhost:27017``.

Go to the MongoDB Download Center website.

- Select "MongoDB Community Server" and download the installer for your OS (Windows/Mac).
 - Follow the installation wizard. On Windows, choose "Install MongoDB as a Service" so it runs automatically in the background.
2. MongoDB Compass (The GUI): A visual tool to look inside your database without the command line. Connect using the default URI: ``mongodb://localhost:27017``.
- This is a GUI (Graphical User Interface) that allows you to visually look inside your database, view collections, and edit documents without using the command line.
 - It usually installs automatically with the Community Edition. If not, download it separately from the MongoDB website.
 - To connect, open Compass, leave the URI as the default ``mongodb://localhost:27017``, and click 'Connect'.

7. Connecting MongoDB to Node.js (Native Driver)

To use MongoDB in a Node.js app without Mongoose, we use the official native driver.

Command to install: ``npm install mongodb``

Basic Native Driver CRUD Code

Code:

```
const { MongoClient } = require('mongodb');

// Connection URI and Database Name
const uri = 'mongodb://localhost:27017';
const client = new MongoClient(uri);

async function runDB() {
  try {
    await client.connect();
    console.log("Connected successfully to MongoDB");

    const db = client.db('SchoolDB');
    const collection = db.collection('students');

    // 1. CREATE (insertOne & insertMany)
    await collection.insertOne({ name: 'Bob', age: 20 });
    await collection.insertMany([
      { name: 'Charlie', age: 21 },
      { name: 'Diana', age: 22 }
    ]);

    // 2. READ (find)
    // .find() returns a cursor, we use .toArray() to
    convert it to a readable JS array
    const students = await collection.find({}).toArray();
    console.log("All Students:", students);

    // 3. UPDATE (updateOne)
    // updateOne takes a filter object, and an update
    object using the $set operator
    await collection.updateOne({ name: 'Bob' }, { $set: {
    age: 25 } });

    // 4. DELETE (deleteOne)
    await collection.deleteOne({ name: 'Charlie' });

  } catch (err) {
    console.error(err);
  } finally {
    await client.close();
  }
}

runDB();
```

8. MongoDB Query Operators

Operators let you perform complex searches in your database.

Comparison Operators

- `$eq`: Equal to
- `$ne`: Not equal to
- `$gt`: Greater than
- `$gte`: Greater than or equal to
- `$lt`: Less than
- `$lte`: Less than or equal to
- `$in`: Matches any of the values specified in an array

Code:

```
// Find students older than 20
collection.find({ age: { $gt: 20 } }).toArray();
// Find students whose age is either 20 or 22
collection.find({ age: { $in: [20, 22] } }).toArray();
```

Logical Operators

- `$and`: Joins query clauses with a logical AND
- `$or`: Joins query clauses with a logical OR

Code:

```
// Find students who are older than 20 OR named Alice
collection.find({ $or: [{ age: { $gt: 20 } }, { name: 'Alice'
}] }).toArray();
```

Sorting, Limit, and Skip (Pagination)

- `sort()`: Orders the result. 1 is ascending, -1 is descending.
- `limit()`: Restricts the number of documents returned.
- `skip()`: Skips a specified number of documents (useful for Page 2, Page 3, etc.).

Code:

```
// Get 5 students, skip the first 10, sort by age descending
collection.find({}).sort({ age: -1
}).skip(10).limit(5).toArray();
```

9. Introduction to Mongoose

What is it? Mongoose is an Object Data Modeling (ODM) library for MongoDB and Node.js.

Why use it? While MongoDB is "schemaless", in real-world apps, we usually WANT a strict schema to prevent bad data from being saved (e.g., stopping a user from saving a string instead of a number for an age). Mongoose enforces schemas, provides built-in validation, and makes writing queries easier.

Installation: `npm install mongoose`

Connecting with Mongoose

Code:

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/SchoolDB')
  .then(() => console.log('Mongoose connected!'))
  .catch(err => console.log('Connection failed:', err));
```

10. Mongoose Schemas, Models & Validation

- **Schema:** The blueprint. It defines exactly what fields are allowed, their data types, and their validation rules.
- **Model:** The compiled version of the Schema. The Model provides the interface (the methods) to interact with the database (like `Model.find()`, `Model.create()`).

Common Validation Options Table

Validator	Data Type	Description
<code>required: true</code>	Any	Forces the field to be mandatory.
<code>min / max</code>	Number	Sets the minimum or maximum numeric value.
<code>minlength / maxlength</code>	String	Sets the string character length limits.
<code>enum</code>	String	Only allows values from a predefined array of strings.
<code>unique: true</code>	Any	Ensures no duplicate values exist in the collection (creates an index).

Example: Creating a Validated Schema and Model

Code:

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
```

```

        required: true,
        minlength: 3
    },
    age: {
        type: Number,
        min: 18,
        max: 65
    },
    role: {
        type: String,
        enum: ['admin', 'user', 'guest'],
        default: 'user'
    }
});

// Create the Model (Mongoose automatically pluralizes 'User'
// to a 'users' collection)
const User = mongoose.model('User', userSchema);

```

11. Full CRUD Operations using Mongoose

Here is a complete, runnable script showing how to Create, Read, Update, and Delete using a Mongoose model.

```

Code:
const mongoose = require('mongoose');

// 1. Connect
mongoose.connect('mongodb://localhost:27017/MyTestDB')
    .then(() => console.log('Connected to DB'))
    .catch(err => console.log(err));

// 2. Define Schema & Model
const studentSchema = new mongoose.Schema({
    name: { type: String, required: true },
    grade: String
});
const Student = mongoose.model('Student', studentSchema);

// 3. CRUD Functions
async function performCRUD() {
    try {
        // CREATE
        const newStudent = await Student.create({ name:
'Umar', grade: 'A' });

```

```

        console.log('Created:', newStudent);

        // READ (Find all)
        const allStudents = await Student.find();
        console.log('All Students:', allStudents);

        // READ (Find one specific)
        const singleStudent = await Student.findOne({ name:
'Umar' });

        // UPDATE
        // { new: true } returns the updated document rather
than the old one
        const updatedStudent = await
Student.findByIdAndUpdate(
            singleStudent._id,
            { grade: 'A+' },
            { new: true }
        );
        console.log('Updated:', updatedStudent);

        // DELETE
        const deletedStudent = await
Student.findByIdAndDelete(singleStudent._id);
        console.log('Deleted:', deletedStudent);

        } catch (error) {
            console.error('Error during CRUD:', error);
        }
    }

performCRUD();

```